

# PRACTICAL NO. 4

## AIM

1. Adam is working in an IT company. He has been given a task to reduce the load of a system by killing some of the processes running in the LINUX operating system. Which commands will he use to complete the given task with the help of the following operation?

- Kill Processes by name
- Kill a process based on the process name
- Kill a single process at a time with the given process ID

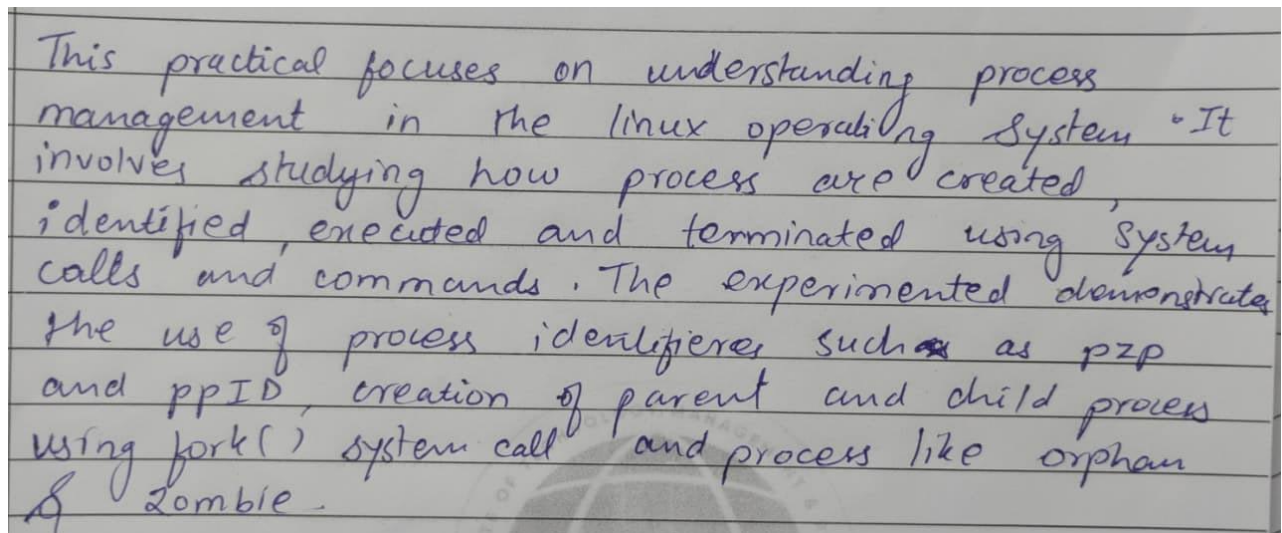
2. Write a program for process creation using C

- Orphan Process
- Zombie Process

3. Create the process using fork () system call.

- Child Process creation
- Parent Process creation
- PPID and PID

## THEORY



This practical focuses on understanding process management in the linux operating system. It involves studying how process are created, identified, executed and terminated using system calls and commands. The experiment demonstrates the use of process identifiers such as pzp and ppID, creation of parent and child process using fork() system call and process like orphan & zombie.

## PERFORMANCE

1.

- Kill Processes by name
- Kill a process based on the process name
- Kill a single process at a time with the given process ID

## OUTPUT

```
m309@m309-BY-OEM:~$ ps -ef
UID          PID    PPID  C STIME TTY          TIME CMD
root           1         0  0  16:19 ?        00:00:07 /sbin/init splash
root           2         0  0  16:19 ?        00:00:00 [kthreadd]
root           3         2  0  16:19 ?        00:00:00 [pool_workqueue_release]
root           4         2  0  16:19 ?        00:00:00 [kworker/R-rcu_gp]
root           5         2  0  16:19 ?        00:00:00 [kworker/R-sync_wq]
root           6         2  0  16:19 ?        00:00:00 [kworker/R-kvfree_rcu_reclaim]
root           7         2  0  16:19 ?        00:00:00 [kworker/R-slub_flushwq]
root           8         2  0  16:19 ?        00:00:00 [kworker/R-netns]
root          10         2  0  16:19 ?        00:00:00 [kworker/0:0H-events_highpri]
root          13         2  0  16:19 ?        00:00:00 [kworker/R-mm_percpu_wq]
root          14         2  0  16:19 ?        00:00:00 [rcu_tasks_kthread]
root          15         2  0  16:19 ?        00:00:00 [rcu_tasks_rude_kthread]
root          16         2  0  16:19 ?        00:00:00 [rcu_tasks_trace_kthread]
root          17         2  0  16:19 ?        00:00:00 [ksoftirqd/0]
root          18         2  0  16:19 ?        00:00:00 [rcu_preempt]
root          19         2  0  16:19 ?        00:00:00 [rcu_exp_par_gp_kthread_worker/0]
root          20         2  0  16:19 ?        00:00:00 [rcu_exp_gp_kthread_worker]
root          21         2  0  16:19 ?        00:00:00 [migration/0]
root          22         2  0  16:19 ?        00:00:00 [idle_inject/0]
root          23         2  0  16:19 ?        00:00:00 [cpuhp/0]
root          24         2  0  16:19 ?        00:00:00 [cpuhp/1]
root          25         2  0  16:19 ?        00:00:00 [idle_inject/1]
root          26         2  0  16:19 ?        00:00:00 [migration/1]
root          27         2  0  16:19 ?        00:00:00 [ksoftirqd/1]
root          29         2  0  16:19 ?        00:00:00 [kworker/1:0H-events_highpri]
root          30         2  0  16:19 ?        00:00:00 [cpuhp/2]
root          31         2  0  16:19 ?        00:00:00 [idle_inject/2]
root          32         2  0  16:19 ?        00:00:00 [migration/2]
```

```
m309@m309-BY-OEM:~$ ps -ef | grep firefox
m309          7713      7697  0 17:10 pts/0        00:00:00 grep --color=auto firefox
m309          7713      7697  0 17:10 pts/0        00:00:00 grep --color=auto firefox
m309@m309-BY-OEM:~$ kill 7713
bash: kill: (7713) - No such process
bash: kill: (7713) - No such process
m309@m309-BY-OEM:~$ pkill firefox
```

2. Write a program for process creation using C

- Orphan Process
- Zombie Process

## DISCRIPTION

*Orphan process*

An orphan process is a child process whose parent process terminates before child finishes execution. The orphan process is adopted by the init or system process.

## Zombie Process

A zombie process is a child process that has completed execution but still remains in the process table because its parent has not read its exit status.

### OUTPUT

#### 1. ORPHAN PROCESS

```
m309@m309-BY-OEM:~$ Child PID: 8566  
New Parent PID: 2119
```

```
m309@m309-BY-OEM:~$ nano orphan.c
```

```
#include <stdio.h>  
#include <unistd.h>  
  
int main() {  
    if (fork() == 0) {  
        sleep(5);  
        printf("Child PID: %d\n", getpid());  
        printf("New Parent PID: %d\n", getppid());  
    } else {  
        printf("Parent exiting\n");  
    }  
    return 0;  
}
```

```
New Parent PID: 2119
```

```
m309@m309-BY-OEM:~$ nano orphan.c  
m309@m309-BY-OEM:~$ gcc orphan.c -o orphan  
m309@m309-BY-OEM:~$ ./orphan  
Parent exiting  
m309@m309-BY-OEM:~$ Child PID: 9309  
New Parent PID: 2119
```

## 2. ZOMBIE PROCESS

```
m309@m309-BY-OEM:~$ nano zombie.c
```

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // Child process
        printf("Child exiting\n");
        return 0;
    } else {
        // Parent process
        printf("Parent sleeping, child becomes zombie\n");
        sleep(30); // VERY IMPORTANT
    }
    return 0;
}
```

[ Read 17 lines ]

Help Write Out Where Is Cut Execute Location M-U Undo M-A Set M

```
m309@m309-BY-OEM:~$ nano zombie.c
m309@m309-BY-OEM:~$ gcc zombie.c -o zombie
m309@m309-BY-OEM:~$ ./zombie
Parent sleeping, child becomes zombie
Child exiting
```

3. Create the process using fork ()system call.

- Child Process creation
- Parent Process creation
- PPID and PID

## DISCRIPTION

### Process

A process is an instance of a program that is currently being executed in the operating system. It includes program code, data stack, and system resources.

### Process ID (PID)

Process ID (PID) is a unique numerical identifier assigned by the operating system to each running process for identification and management.

### Parent process

A parent process is a process that creates one or more child processes using system calls such as `fork()`.

### Child process

A child process is a newly created process that is generated by a parent process and executes independently.

### Parent process ID (PPID)

PPID shows the process ID of the parent of a running process.

## OUTPUT

```
m309@m309-BY-OEM:~$ nano fork_demo.c
m309@m309-BY-OEM:~$ gcc fork_demo.c -o fork_demo
```

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());
    } else {
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());
    }
    return 0;
}
```

Help Write Out Where Is Cut Execute Location M-U Undo M-A Set M

```
m309@m309-BY-OEM:~$ gcc fork_demo.c -o fork_demo
m309@m309-BY-OEM:~$ ./fork_demo
Parent exiting
m309@m309-BY-OEM:~$ Child PID: 8566
New Parent PID: 2119
```

## 4.INFINITE LOOP PROCESS

Infinite loop process is a process that runs continuously without termination until it is manually stopped by the user or system.

## OUTPUT

[illegible]

```
#include <stdio.h>
#include <unistd.h>

int main() {
    while (1) {
        printf("Running...\n");
        sleep(1);
    }
    return 0;
}
```

```
Running...  
Running...  
Running...  
Running...  
Running... m309@m309-BY-OEM:~$ q  
Running... Command 'q' not found, but can be installed with:  
Running... sudo snap install q # version 1.6.3-1, or  
Running... sudo apt install python3-q-text-as-data # version 3.1.6-3  
Running... See 'snap info q' for additional versions.  
Running... m309@m309-BY-OEM:~$
```